```c
int main () {
    int x = 15;
    int v[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    v[5] = x + v[8];
}
```

Sample Solution

```
.data
x: .word 15
v: .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

.text
main:
    # Load 15 into reg s0
    la t0, x              # t0 = address in x
    lw s0, 0(t0)          # s0 = 15

    # Get value in v[8] to reg t1
    la s1, v              # s1 = v[]
    lw t1, 32(s1)         # t1 = v[8]

    # Add and store results in v[5]
    add t2, s0, t1        # t2 = x + v[8]
    sw t2, 20(s1)         # v[5] = t2

    # Exit
    li a7, 10
    ecall
```

Function Calls

- **Caller:** calling function (in this case, `main`)
- **Callee:** called function (in this case, `sum`)

C Code

```
void main()
{
  int y;
  y = sum(42, 7);
  ...
}

int sum(int a, int b)
{
  return (a + b);
}
```

- **Caller:**
  - passes **arguments** to calle
  - jumps to callee
- **Callee:**
  - **performs** the function
  - **returns** result to caller
  - **returns** to point of call
  - **must not overwrite** regist memory needed by caller

Input Arguments & Return Value

C Code

```
int main()
{
  int y;
  ...
  y = diffofsums(2, 3, 4, 5);  // 4 arguments
  ...
}

int diffofsums(int f, int g, int h, int i)
{
  int result;
  result = (f + g) - (h + i);
  return result;                // return value
}
```

Northeastern University 4

RISC-V Function Calls

- **Call Function:** jump and link (`jal func`)
- **Return** from function: jump register (`jr ra`)
- **Arguments**: `a0 - a7`
- **Return value**: `a0`

C Code

```
int main() {
  simple();
  a = b + c;
}

void simple() {
  return;
}
```

RISC-V assembly code

```
0x00000300 main: jal  simple
0x00000304       add  s0, s1,
...

0x0000051C simple: jr ra
```

jal: jumps to `simple` label (PC = 0x0000051C)
`ra` = PC + 4 = 0x00000304

jr ra: jumps to address in `ra` (0x00000304)

void means that s doesn't return a va

Input Arguments & Return Valu

RISC-V assembly code

```
# s7 = y

main:
  ...
  addi a0, zero, 2    # argument 0 = 2
  addi a1, zero, 3    # argument 1 = 3
  addi a2, zero, 4    # argument 2 = 4
  addi a3, zero, 5    # argument 3 = 5
  jal  diffofsums     # call Function
  add  s7, a0, zero   # y = returned value
  ...

# s0 = result
diffofsums:
  add t0, a0, a1      # t0 = f + g
  add t1, a2, a3      # t1 = h + i
  sub s3, t0, t1      # result = (f + g) - (h + i)
  add a0, s3, zero    # put return value in a0
  jr  ra              # return to caller
```

Northeastern University

Simple Function Calls Exercise

Implement the main function below with RISC-V assemb language:

```
int sum(int a, int b) {
    return a + b;
}
int main() {
      int a = 5;
      int b = 10;
      // Calculate sum
      int result = sum(a, b);
}
```

```
  .data
int_a: .word 5
int_b: .word 10

  .text
main:
# Continue from here...
```

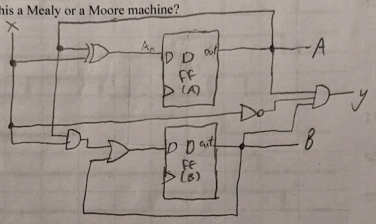
Northeastern University

3) (8 pts.) A sequential circuit consists of two D flip-flops (A and B). The circuit has one external input x, and one external output y. The following equations describe the next state equations for the two flip-flops (A_n for A, B_n for B), as well as the output equation for output y.

$$A_n = A \oplus x$$
$$B_n = Ax + B$$
$$y = AB\bar{x}$$

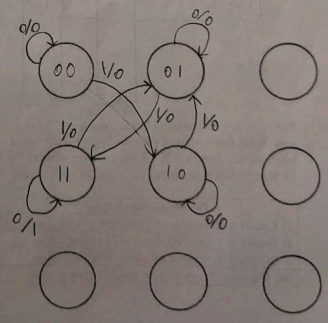
a.) Draw the sequential logic circuit diagram.
b.) Complete the state table (truth table) for the circuit above.
c.) Complete the state machine diagram (We have provided nine states. You will not need all of them. Use only as many as you need and label them accordingly).
d.) Is this a Mealy or a Moore machine?

a)



b)

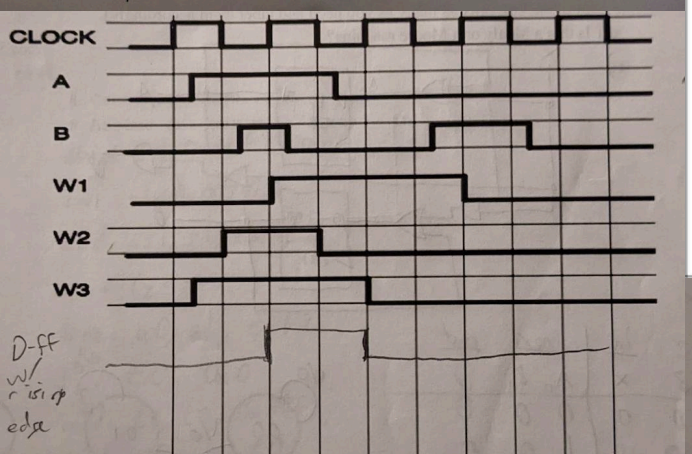
Curr A	Curr B	in x	next A_n	next B_n	out y
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	0	1	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	0	1	0
1	1	0	1	1	1
1	1	1	0	1	0



d) This is a Mealy machine because output depend on input and state


```c
int main()
{
    int i, j, k;
    cout << "Please enter the first value to add: ";
    cin >> i;
    cout << "Please enter the second value to add: ";
    cin >> j;
    k = i + j;
    cout << "The result is " << k;
}
```

```
.data
    prompt1: .string "Please enter the first value to add: "
    prompt2: .string "Please enter the second value to add: "
    result: .string "The result is "
.text
main:
    # Register conventions
    # i is s0
    # j is s1
    # k is s2

    # register int i =
    # input("Please enter the first value to add: ");
    addi a7, zero, 4
    la a0, prompt1
    ecall

    addi a7, zero, 5
    ecall
    mv s0, a0

    # register int j =
    # input("Please enter the second value to add: ");
    addi a7, zero, 4
    la a0, prompt2
    ecall
    addi a7, zero, 5
    ecall
    mv s1, a0

    # register int k = i + j;
    add s2, s1, s0

    # print("The result is " + k);
    addi a7, zero, 4
    la a0, result
    ecall
    addi a7, zero, 1
    mv a0, s2
    ecall

    # End the program
    li a7, 10
    ecall
```



Flip-Flop or Latch Type	*W1*	*W2*	*W3*	*NONE*
D Latch with **enable** (*Clock signal is the enable*)			X	
D flip-flop Triggered on clock's **rising edge**				X
T flip-flop Triggered on clock's **falling edge**		X		
JK flip-flop Triggered on clock's **rising edge**	X			